KeyTrapDiscovery_ChatGPTo1_V2

You are an DNS security expert. Here is an example attack that caused by unbounded number of operations. Learn this attack carefully.

Example:

NRDelegation attack

Summary:
The core mechanism of the NRDelegation Attack involves a malicious client sending a specially crafted query to a DNS resolver. This query is engineered in such a way that it forces the resolver to contact a malicious authoritative DNS server. This server then sends a referral back to the resolver pointing towards additional DNS servers that do not respond. The recursive nature of these queries, coupled with non-responsive servers, creates a chain of requests that consumes significant resources from the resolver, particularly taxing its CPU and memory. This leads to an exhaustion of the resolver's resources, making it unable to serve legitimate requests, effectively denying service.

The attack differs from traditional DDoS attacks on DNS systems, which typically rely on amplification techniques—where a small query triggers a large volume of responses. Instead, the NRDelegation Attack causes a series of complex and resource-intensive operations within the DNS infrastructure, making it a more stealthy and effective form of attack.

Detailed breakdown of the attack:

Initiation: A malicious client sends a DNS query to a target recursive resolver. This query is specially crafted to initiate a chain of referrals.

Malicious Referral: The query is redirected to a malicious authoritative DNS server, which responds with a referral to

another set of DNS servers. Crucially, these referred servers are non-responsive.

Resource Exhaustion: Each referral forces the resolver to attempt resolving the provided nameservers, which do not respond. The resolver expends CPU and memory resources trying to handle these referrals, which are crafted to cause maximum recursion or looping within the resolver’s processing logic.

Amplification of Attack: Because the non-responsive servers do not answer, the resolver continues to retry, significantly amplifying the CPU and memory load. This is designed to exhaust the resolver's resources, preventing it from serving legitimate requests.

Here's a detailed explanation of why non-responsive referrals consume significant computational power:

DNS Referral Process: When a DNS resolver receives a query, if the answer isn't in its cache, it must query authoritative servers. Sometimes, these servers respond with referrals to other servers rather than the answer. This involves delegating the query to other name servers.

Expensive Memory and Cache Access: Each referral requires the resolver to check if the referred domain names are already resolved in its cache or if they need further resolution. This checking process involves extensive memory access. When the resolver receives referrals to servers that do not respond, it cannot complete the resolution process quickly and must retry or escalate through more referrals, repeatedly accessing its cache and memory.

Restarting Resolution Process: Each non-responsive referral forces the resolver to restart the resolution process for the same query. In the case of the NRDelegation Attack, the resolver encounters multiple referrals that lead nowhere, causing it to restart multiple times without progress.

Amplifying Resource Consumption: The attack cleverly makes the resolver's job not just iterative but exponential in some cases, by causing repeated restarts and checks against the cache and memory for each referral in the chain. This significantly increases CPU and memory usage because each iteration is not just a simple repeat but

Main reason of the attack and more explanation:

The NRDelegation Attack involves a malicious client triggering the DNS resolver to perform an unbounded number of operations for a single client request, which is one of the central reasons for the significant computational load imposed by the attack. Here’s a detailed explanation based on the paper's content:

Chain Reaction of Requests: The attack exploits the referral process, where a malicious authoritative server responds to a query with referrals to other DNS servers. These referrals are specifically crafted to servers that do not respond (non-responsive or NR).

Resolver Restart Mechanism: Each time the resolver receives a referral to a non-responsive server, it must restart the resolution process for the domain, ignoring previously established resolution states, including any safety limits like retry counters. This mechanism is critical because each restart requires the resolver to perform a full set of operations from the beginning, dramatically increasing the computational burden.

Ignoring Safety Counters: The recursive resolver, during an NRDelegation attack, ignores some of its built-in safety mechanisms that would typically prevent unbounded operation chains. These safety mechanisms are designed to stop the resolver from entering into infinite loops or overly lengthy resolution processes under normal operational conditions.

Excessive Memory and CPU Utilization: Due to

continuous restarts and the need to resolve each referral anew, the resolver engages in extensive memory and CPU usage. Each referral cycle causes significant resource consumption as the resolver checks its cache and attempts to contact the referred DNS servers, which do not respond.

Lack of Response from Referred Servers: The non-responsive nature of the servers to which the resolver is referred means that each referral ends without resolving the query, thereby forcing another cycle of referrals and restarts, repeating indefinitely or until system limits are finally engaged.

The attack effectively turns a single malicious query into an ongoing process that exhausts the resolver's resources, preventing it from servicing legitimate queries and leading to a denial of service. This is a sophisticated method that exploits the fundamental workings of DNS resolvers, particularly how they handle domain resolution and referrals, making it a potent and stealthy form of attack.

I want to you find similar computational complexity attacks based on DNSSEC.

Below are some *conceptual* examples of how DNSSEC (Domain Name System Security Extensions) can be misused to launch “computational complexity” or resource-exhaustion attacks, in a manner similar to the NRDelegation Attack. While many of these scenarios are theoretical or exploit misconfigurations, they illustrate how DNSSEC’s cryptographic validations and complex record chains can be manipulated to consume excessive CPU, memory, or network resources on a resolver.

---

1. NSEC3 “High Iteration” Attacks

Overview

NSEC3 is an alternative to NSEC for providing authenticated denial of existence. It uses a hash of the domain name, along with iteration parameters and salt, to prevent zone walking.

However, if the iteration count is set excessively high (within the permitted limits in the zone), it forces DNS resolvers to perform multiple hash computations for every negative response validation.

Attack Vector

1. **Crafted Query**: A malicious client repeatedly queries non-existent names in a signed zone that uses NSEC3 with a high iteration count.
2. **NSEC3 Lookup**: Each negative response must include an NSEC3 record. The resolver then *must* validate the NSEC3 record, which involves:
    - Multiple hash computations (based on the iteration parameter).
    - Signature validation of the NSEC3 record (cryptographic operations).
3. **CPU Exhaustion**: Because each negative response triggers potentially large numbers of hashing iterations plus signature validation, the resolver's CPU usage spikes, potentially leading to denial of service if enough queries are made or the iteration count is very high.

Key Points

- **Hash Iterations**: The higher the iteration count, the more computationally expensive the validation.
- **Cryptographic Signature Checks**: Each NSEC3 record is signed, and signature checks also use non-trivial CPU resources.
- **Repeated Queries**: Attackers can send many queries for non-existent names, each generating a negative answer that triggers hashing and signature validation overhead.

---

2. Overly Large DNSKEY / DS Chains

Overview

DNSSEC relies on a chain of trust from the root down to the authoritative zone. The resolver must fetch and validate:

- DNSKEY records (public keys),
- DS records (delegation signer records),

- RRSIG records (signatures over the keys/zone records).

If a zone (or a chain of zones) publishes excessively large DNSKEY sets or has unusually large DS records, the resolver may be forced to perform multiple expensive cryptographic operations (signature verification, key parsing, etc.).

Attack Vector

1. **Malicious or Misconfigured Zone**: An attacker (who controls a zone) publishes DNSKEY records in bulk (e.g., tens or hundreds of keys) or uses unnecessarily large RSA keys (within RFC-permitted limits).
2. **Complex Delegations**: They also create a chain of delegations where each delegation has its own large DS set, referencing multiple algorithms or multiple keys.
3. **Repeated Validations**: Each time the resolver has to validate the chain of trust (for example, if a record is not in cache or if TTLs are short), it must:
    - Retrieve all DNSKEYs,
    - Retrieve DS records in the parent zone,
    - Verify multiple RRSIGs cryptographically.

Key Points

- **Large DNSKEY Sets**: Each DNSKEY must be verified and can consume significant CPU time, especially for large key sizes or expensive algorithms (e.g., RSA with large modulus).
- **Multiple DS Records**: Each DS has to be processed, especially if they use multiple algorithms (e.g., DS with SHA-256, SHA-384, etc.).
- **Frequent Revalidations**: Attackers can force revalidation by sending queries for many subdomains or using short TTLs so that cache entries expire quickly.

---

3. Recursive Signature Chaining / Cross-Signed Zones

Overview

DNSSEC signatures can cross-reference multiple keys, particularly in scenarios involving key rollovers or cross-signed child-parent relationships. If misused, this can result in a tangle of

references that the resolver must repeatedly fetch and validate.

Attack Vector

1. **Complex Key Rollovers**: During a key rollover, a zone might temporarily publish multiple keys (old key, new key) and sign zone data with both, or keep older DS references in the parent.
2. **Cross-Signed Zones**: Zones can be arranged so that each “child” zone includes references to multiple “parent” keys or extraneous DS records. This can create a labyrinth of extra DNSKEY / DS fetches.
3. **Repeated Lookups**: A malicious client queries records that force the resolver to follow the entire cross-signed chain, verifying multiple RRSIGs at each step. If some references are invalid or non-responsive, the resolver may restart portions of the validation process repeatedly.

Key Points

- **Multiple RRSIG Validations**: Each record (DNSKEY, DS, zone data) may have multiple signatures that must be verified.
- **Dead Ends or Restarted Validation**: Incorrect or non-responsive references in the chain can force the resolver to repeat validation steps, similar to how NRDelegation triggers multiple restarts.
- **Excessive CPU Usage**: Combined cryptographic checks on multiple keys and multiple signatures degrade resolver performance significantly.

---

4. CNAME / DNAME Chaining with DNSSEC Validation

Overview

DNSSEC-signed CNAME or DNAME records introduce another layer of validation overhead. In principle, CNAME or DNAME is supposed to be straightforward, but if an attacker sets up *chained* or *looping* CNAME/DNAME references in combination with signed responses, the resolver may do extra work.

Attack Vector

1. **Chained CNAME**: A malicious zone publishes multiple CNAME records pointing to each other or to multiple next-level queries that all have their own signed records.
2. **DNAME Complexity**: Similarly, a DNAME can redirect entire sub-trees, causing the resolver to fetch multiple DNSSEC-signed responses to validate the chain of redirections.
3. **Signature Checks for Each Hop**: Each new alias or redirection requires the resolver to fetch the corresponding DNSKEY, DS, and RRSIG, verifying them cryptographically.

Key Points

- **Multiple Hops**: Each hop in the CNAME or DNAME chain requires a separate DNSSEC validation step.
- **Potential Loops**: If misconfigured, loops can arise, forcing repeated resolution attempts until the resolver's loop detection or restart logic is triggered.
- **Amortized CPU Usage**: Even if loops are eventually caught, the overhead from cryptographic validation can be large enough to degrade service.

---

5. "Fallback" or "Algorithm Agility" Exhaustion

Overview

DNSSEC supports multiple cryptographic algorithms for signing (RSA, ECDSA, EdDSA, etc.) and hashing (SHA-1, SHA-256, SHA-384, etc.). A resolver may attempt multiple validation paths if it is unsure which algorithms the zone or parent is using (particularly during transitions).

Attack Vector

1. **Multiple Algorithms Listed**: A zone can include DS records specifying multiple hash algorithms and DNSKEY records using multiple signing algorithms.
2. **Malicious Queries**: Attackers can send queries that trigger the resolver to test multiple validation paths, especially if the zone or its parent uses "algorithm agility" incorrectly.
3. **Repeated Signature Failure and Retry**: Each time a signature validation fails, the resolver may attempt the next algorithm, or re-check DNSKEY sets, leading to extra CPU use.

Key Points

- **Excessive Algorithm Handling**: Each algorithm requires different cryptographic operations.
- **Repeated Failures**: If the zone is intentionally misconfigured, the resolver may loop through or repeatedly attempt each algorithm.
- **Time-Consuming Operations**: Some algorithms (especially large-key RSA) are very CPU expensive.

Mitigations and Best Practices

1. **Limit NSEC3 Iterations**: RFCs provide guidance on a maximum iteration count. Operators should avoid excessively high iteration settings.
2. **Enforce Query/Fetch Limits**: Similar to how resolvers limit recursion depth, ensure your resolver enforces a maximum number of restarts or queries per resolution.
3. **Cache Negative Responses**: Negative caching can reduce repetitive NSEC/NSEC3 lookups for the same non-existent domains.
4. **Rate Limiting**: Implement query rate limiting or use response rate limiting to cut off excessive requests from a single source.
5. **Validate Properly Signed Zones**: Ensure your resolver strictly adheres to DNSSEC protocol rules and does not repeatedly attempt to validate obviously broken signatures or zones.
6. **Monitor and Tune Timeouts**: Shorten timeouts for unresponsive name servers or broken DS/DNSKEY references to reduce the impact of repeated retries.
7. **Algorithm Choice**: Use modern, efficient algorithms (like ECDSA or EdDSA) rather than large RSA keys, reducing the CPU overhead for validation.

Conclusion

DNSSEC provides authenticity and integrity for DNS data but at the cost of additional complexity and cryptographic overhead. Attackers can exploit this overhead—and potential misconfigurations—to craft “computational complexity” attacks that consume disproportionate CPU, memory, or bandwidth on resolvers. By carefully managing NSEC3 parameters, DNSKEY sizes, algorithm agility, and resolution restarts, DNS operators can mitigate these risks and keep DNS resolvers resilient against such stealthy denial-of-service techniques.

Large DNSKEY Sets: Each DNSKEY must be verified and can consume significant CPU time, especially for large key sizes or expensive algorithms (e.g., RSA with large modulus).

Focus on large number of DNSKEY approach. Elaborate on this

Below is an expanded discussion on how publishing a large number of DNSKEY records (especially with costly algorithms or large key sizes) can be weaponized to overwhelm DNS resolvers. This is sometimes called a “DNSSEC key flood” or “DNSKEY-bloat attack,” and it focuses on how DNSSEC’s chain of trust and per-key validation overhead can be exploited to create a computational complexity attack.

How Large DNSKEY Sets Cause Excessive Load

1. **Multiple Keys = Multiple Signatures**
    - Each DNSKEY record in a zone is typically accompanied by an RRSIG (signature) over the DNSKEY set itself.
    - A resolver must verify each RRSIG with the corresponding public key (DNSKEY).
    - If a zone operator publishes, for example, dozens (or even hundreds) of DNSKEY records, the resolver must perform dozens (or hundreds) of signature verifications whenever it fetches the DNSKEY RRset.
2. **Cryptographic Costs of Validation**
    - DNSSEC validation involves cryptographic operations (RSA, ECDSA, EdDSA, etc.).

- For large-key RSA (e.g., 2048-bit or 4096-bit), each signature validation can be slow relative to typical DNS lookups.
- Even with more efficient elliptic-curve algorithms, a large quantity of signatures can create a significant cumulative CPU load.

3. **Impact of Repeated Fetches**
    - Attackers can force resolvers to refetch the DNSKEY RRset frequently (e.g., by querying multiple subdomains or by exploiting short TTLs).
    - Every time the resolver's cache entry for the DNSKEY set expires, the resolver must re-verify all the keys and signatures again for subsequent queries to that domain or subdomains.
    - If the attacker alternates between multiple subdomains (causing partial cache misses) or uses high query volume, the resolver spends disproportionate CPU time revalidating the large DNSKEY set.
4. **Chain of Trust Amplification**
    - DNSSEC validation requires not just fetching the child zone's DNSKEY set but also verifying DS records from the parent zone.
    - In scenarios where multiple DS records exist (pointing to multiple DNSKEYs in the child zone), each DS -> DNSKEY path may be validated.
    - Large DNSKEY sets in both the parent and child zones multiply the validation tasks.
5. **Memory Overhead**
    - Storing and parsing large DNSKEY sets also increases memory usage in the resolver.
    - Especially under high concurrency, resolvers handling many parallel queries can encounter memory pressure from storing large amounts of cryptographic material (public keys and signatures).
6. **Loop or Revalidation Triggers**
    - Some resolvers may attempt fallback or retry if they see unexpected or conflicting DS/DNSKEY states.
    - In misconfigured or maliciously configured zones, the presence of extraneous keys can confuse the resolver into attempting multiple validation paths, each of which leads to additional cryptographic checks.

Attack Workflow in a Typical Large-DNSKEY Attack

1. **Zone Preparation (Attacker-Controlled)**
    - The attacker (or malicious zone operator) publishes an extremely large DNSKEY RRset (e.g., tens or hundreds of keys) in their zone file.
    - Each key might use a large RSA modulus (e.g., 4096 bits) or a slow algorithm to maximize CPU cost.
2. **Malicious Queries**
    - The attacker's botnet or attack clients issue a high volume of DNS queries for records in the attacker's zone (or subdomains).
    - Alternatively, a single malicious client can repeatedly force cache misses by randomizing subdomains, bypassing caching mechanisms.
3. **Resolver Fetch and Validation**
    - For each unique subdomain (or once TTLs expire), the recursive resolver fetches the DNSKEY RRset from the malicious zone's authoritative server.
    - The resolver must parse and store each public key, then validate the signatures over the DNSKEY set, often verifying dozens or more RRSIGs.
4. **CPU and Memory Consumption**
    - High-cost signature validations for each key saturate the resolver's CPU.
    - The resolver's memory usage grows as it stores large RRsets and RRSIGs for each query.
    - If the attack volume is sufficient, legitimate DNS queries queue up or fail as the resolver becomes overloaded.
5. **Denial of Service**
    - Eventually, the recursive resolver can no longer process legitimate queries promptly, effectively causing partial or complete DNS outage for the resolver's clients.

Why This Attack Is Effective

- **Stealthiness**: It can appear as normal DNSSEC traffic; the zone is just "using a lot of keys."
- **Bypassing Rate Limits**: Rate-limit defenses that only limit packet-per-second or bandwidth-based attacks may not address the *computational cost* of verifying many large keys.
- **Leverage DNSSEC Requirements**: DNS resolvers are *required* by the protocol to validate DNSSEC-signed answers. Ignoring or refusing to validate an apparently legitimate zone breaks DNSSEC's trust model.

Real-World Considerations and Mitigations

1. **Limit on Maximum DNSKEY RRset Size**
    - Some resolvers implement configuration directives that cap the number of keys in a single RRset they will accept. If a zone publishes more than that cap, the resolver may treat the zone as misconfigured and refuse to validate it.
2. **Key Algorithm Policies**
    - Many enterprises or ISPs use policy that flags very large or out-of-policy keys (e.g., RSA 4096 or certain exotic key types) as suspicious. They might mark the zone as insecure or refuse to validate it to avoid resource exhaustion.
3. **Negative Trust Anchors**
    - If a zone is found to be malicious or misconfigured, operators can install a local "negative trust anchor" telling the resolver to treat it as unsigned (thus skipping validation) or entirely block it. This is a temporary measure until the zone is fixed or the threat passes.
4. **Pre-Validation and Rate Limiting**
    - Some advanced resolvers use heuristics to detect suspiciously large key sets and can limit the CPU time or number of queries allocated to that domain.
    - Rate limiting can be applied at the *validation* stage, so a single domain's keys don't monopolize CPU resources.
5. **Algorithm Choice**

- Encouraging the use of more efficient algorithms (e.g., ECDSA, EdDSA) lowers the CPU cost of each signature validation.
- Operators should discourage or detect large RSA key usage unless there is a compelling security requirement.

6. **Staged Key Rollovers**
    - If a zone truly needs multiple keys (e.g., during rollover), the rollover should be staged and short-lived, so the DNSKEY RRset does not remain bloated for a long period.

Conclusion

A large-DNSKEY attack leverages a fundamental element of DNSSEC—cryptographic validation—to impose excessive CPU and memory usage on DNS resolvers. By publishing an inordinately large number of DNSKEY records (especially with expensive cryptographic parameters), an attacker can cause each validation cycle to become extremely costly. When paired with high query volume or short TTLs, this can quickly degrade or disable a resolver's service.

The best countermeasures rely on both protocol-level defenses (e.g., limiting DNSKEY RRset size, adopting more efficient cryptographic algorithms, negative trust anchors) and operational best practices (e.g., rate limiting, well-managed key rollovers, suspicious domain monitoring). Ultimately, staying vigilant about unusual DNSKEY configurations and enforcing sensible validation limits are essential to mitigating this and similar computational-complexity attacks in DNSSEC.

> For a maximum size DNS reponse, what is the maximum number of keys and signatures we can fit?

Below is a simplified way to estimate the maximum number of DNSKEY RR (resource records) and corresponding RRSIGs that can fit into a *single* DNS response of up to 65,535 bytes—the maximum possible DNS message size. Please note that the exact number depends heavily on (1) the key sizes and algorithms used, (2) how names are compressed, and (3) other

overhead in the DNS message. Hence, these calculations are *approximate* but illustrate the general ballpark.

---

1. Relevant DNS Size Constraints

1. **DNS Message Size Limit**:
   DNS messages can be up to 65,535 bytes in total length (including header, question, answer, authority, and additional sections).
2. **Header and Question Overhead**:
   - **DNS Header**: 12 bytes
   - **Question Section**: Typically 12–20 bytes (depending on the queried name and compression). Let's assume about 20 bytes total for simplicity.
   - This leaves roughly **65,535 − 12 − 20 ≈ 65,503 bytes** for all answer RRs, authority RRs, and additional RRs.
3. **Per-RR Overhead**:
   In the Answer (or Authority) section, each Resource Record has:
   - **Name field**: Often compressed to a 2-byte pointer if all RRs share the same owner name.
   - **Type**: 2 bytes
   - **Class**: 2 bytes
   - **TTL**: 4 bytes
   - **RDLENGTH**: 2 bytes indicating how many bytes of RDATA follow.

   So at minimum, each RR has **(2 + 2 + 2 + 4 + 2) = 12 bytes** of overhead before the actual RDATA.

---

2. Approximate Size of a Single DNSKEY + RRSIG Pair

A DNSKEY record (RDATA) contains:

- Flags (2 bytes)
- Protocol (1 byte, typically 3)
- Algorithm (1 byte)
- Public key data (Base64-encoded)

A typical **2048-bit RSA** DNSKEY has around 270–300 bytes of base64 key material, plus 4–5 bytes of overhead for Flags/Protocol/Algorithm. Let's pick **~280 bytes** for the RDATA portion. After including the 12-byte RR overhead, we get ~292 bytes for a single DNSKEY record.

Similarly, an RRSIG (which signs the DNSKEY set) contains:

- RRSIG header fields (type covered, algorithm, labels, original TTL, signature expiration, etc. ~ 18–20 bytes)
- Signature data (Base64-encoded). For a 2048-bit RSA signature, this might be ~256–300 bytes.

Hence, an RRSIG over a 2048-bit RSA DNSKEY might have **~280–320 bytes** of RDATA. Including the 12-byte RR overhead, an RRSIG can easily be **~300–330 bytes** total. Let's use **~310 bytes** as a rough average.

Combined: One DNSKEY + its corresponding RRSIG might total in the ballpark of **(292 + 310) ≈ 602 bytes** per "key pair."

(Note: Real implementations can vary by tens of bytes depending on algorithm specifics, key sizes, base64 length, and so on.)

---

3. How Many DNSKEY+RRSIG Pairs Fit into 65,503 Bytes?

If each pair consumes ~600 bytes, then:

$$\frac{65{,}503 \text{ bytes available}}{600 \text{ bytes per DNSKEY+RRSIG pair}} \approx 109 \text{ pairs}$$

However, this **does not** account for:

- Possibly an SOA record or other RRs in the message.
- Slightly larger RDATA for bigger keys (4096-bit RSA can be almost twice as large).

- Additional overhead if there are multiple RRSIGs per DNSKEY (e.g., multi-algorithm signing).
- Buffer space for name compression details, EDNS(0) options, etc.

In practice, you might see anywhere from **80 to 100+** pairs if you really push the 65 KB boundary with “medium” (2048-bit) RSA keys. With larger keys (e.g., 4096-bit RSA), that number might drop to **30–50**. Conversely, with smaller or more efficient elliptic-curve algorithms (e.g., ECDSA P-256, Ed25519), each key/signature is smaller, allowing more pairs to fit into a single response.

---

4. Real-World Considerations

1. **EDNS(0) and Fragmentation**
   - Although DNS theoretically allows up to 65,535 bytes, large responses over UDP often get fragmented or force TCP fallback. Many resolvers and networks limit effective UDP payload (for instance, 4096 bytes in EDNS). Large RRsets often end up requiring TCP anyway.
2. **Multiple RRSIGs per DNSKEY RRset**
   - Zones that sign with more than one algorithm or during rollover can publish multiple RRSIGs over the same DNSKEY set, multiplying the total size.
3. **Operational Limits**
   - Many DNS operators avoid letting DNSKEY RRsets become this large, because it’s detrimental to performance (and can look suspicious).
   - Resolvers or security policies often cap the number of DNSKEY records they will accept to avoid resource-exhaustion or “DNSKEY-bloat” attacks.
4. **Algorithm Efficiency**
   - Elliptic-curve algorithms (ECDSA, EdDSA) produce much smaller keys and signatures than RSA 2048/4096, so you can pack more of them into a single 65 KB response. This also reduces CPU overhead for validation.

Bottom Line

Under **idealized** conditions (single UDP/TCP DNS response maxing out at ~65 KB, minimal overhead, 2048-bit RSA DNSKEYs), you can typically fit on the order of **80–100 DNSKEY+RRSIG pairs** into one DNS response. Larger key sizes or additional overhead (e.g., multiple RRSIGs per key, other mandatory records) will reduce that number, and smaller/efficient algorithms can increase it slightly.

Ok. Let's say 100 DNSKEYs + RRSIGs. What is the number of validations the resolver need to perform for such DNSSEC reply?

Below is a high-level look at how many *signature validations* a resolver **might** perform in a worst-case scenario when confronted with a DNSKEY RRset containing 100 DNSKEY records plus 100 corresponding RRSIGs.

---

1. Why 100 Signatures Might Mean 100 Validations

DNSSEC signatures (RRSIGs) are attached *per RRset*, not per individual record. However, in a pathological or intentionally malicious configuration, **each of those 100 DNSKEYs could be used to sign the entire DNSKEY RRset**, resulting in 100 RRSIGs over the same RRset. In principle, a resolver:

1. **Fetches the DNSKEY RRset** (100 DNSKEYs).
2. **Sees 100 RRSIGs** attached to that same set.
3. Tries to **validate each RRSIG** if it cannot immediately determine which key(s) are "legitimate" or which DS record(s) in the parent zone match.

In a naive or worst-case implementation, the resolver attempts *up to 100 cryptographic signature checks*—one for each RRSIG—to see if it can form a valid chain of trust with any of them.

Why Would the Resolver Try Them All?

- **Multiple DS Records in Parent**: If the parent zone publishes multiple DS records (one for each of the 100 keys), the resolver sees them all as potential trust paths. It may systematically attempt every (DS, DNSKEY, RRSIG) combination to find a valid chain.
- **No Early Stop (Naive Resolver)**: Even if an implementation *could* short-circuit after finding a single valid signature, not all resolvers do so optimally. Some might attempt to verify each signature in turn, or keep retrying if partial checks fail.

---

2. Real-World vs. Theoretical Worst Case

1. **Practical Implementations**
   - Many resolvers do try to short-circuit as soon as they find a valid signature. They also often match DS records directly to a subset of DNSKEYs (by checking the key tag and algorithm).
   - In a “normal” zone, you rarely have more than a few DNSKEYs, and typically only one or two actually sign the DNSKEY set.
2. **Malicious / Pathological Zones**
   - An attacker controlling a zone can bloat the DNSKEY RRset with 100 (or more) keys, each with its own RRSIG.
   - They can also publish matching DS records in the parent, *forcing* the resolver to consider all 100 keys as potentially valid signers.
   - In this worst-case scenario, the resolver may perform 100 RSA or ECC validations—especially if it cannot trivially determine which signatures are bogus without doing the math.
3. **Other Factors**
   - **Algorithm choice**: Large-key RSA (e.g., 2048-bit or 4096-bit) is costlier to validate than ECDSA or EdDSA.
   - **Implementation shortcuts**: A smart resolver might do pre-checks on key tags or perform only the first few validations until one succeeds.
   - **Caching**: Once validated, the DNSKEY RRset is cached; repeated queries within TTL might avoid re-validating all signatures unless the attacker triggers more cache misses.

3. Conclusion: 100 DNSKEYs = Up to 100 Validations

In the *absolute worst-case scenario* (malicious or highly pathological zone design), **100 DNSKEY records and 100 corresponding RRSIGs** over that same DNSKEY RRset can lead a resolver to attempt **100 cryptographic signature validations** just to authenticate one DNSKEY set. This is precisely how attackers can engineer a “computational complexity” or “DNSKEY-bloat” attack using DNSSEC: by forcing the resolver to do a large amount of cryptographic work on every query.

In practice, many resolvers will not fully validate all 100 signatures if they can identify one valid chain quickly. However, if the parent zone’s DS records reference all 100 keys—or if the resolver is naïve—each signature might be attempted, creating a *significant* CPU burden and potential denial-of-service scenario.

> Conclusion: 100 DNSKEYs = Up to 100 Validations
>
> Focus here. Is there way to increase the number validations for this DNSSEC reply? Be create and come up with smarter ideas

Below are some creative ways that a malicious or “pathologically configured” DNSSEC zone might increase the *number of individual signature validations* that a resolver must perform—well beyond a simple 1:1 mapping of “100 DNSKEYs = 100 RRSIGs.” The key idea is to trick or force resolvers into *testing many possible validation paths* or *checking extra signatures*, thereby multiplying CPU load.

---

1. Multiple DS Records from the Parent Zone

- **How it Works**: Publish multiple DS records (potentially one per DNSKEY) in the parent zone.
- **Effect**:
    1. The resolver sees (say) 100 DS records, each referencing a different child DNSKEY by key tag.

2. Even if the child zone's DNSKEY RRset has 100 keys + 100 RRSIGs, the resolver tries to match each DS to its corresponding DNSKEY.
3. In naive implementations, the resolver may attempt to validate *each RRSIG with each matching DS–DNSKEY pair*, rather than short-circuiting once it finds a valid chain.

- **Bonus Trick**: Some DS entries can be partially incorrect (mismatched key tags, wrong hashing algorithms), causing repeated attempts and fallback logic, further ballooning the total validations.

---

2. Multi-Algorithm Signing on the Same RRset

- **How it Works**: Use multiple DNSSEC algorithms *simultaneously* for the same DNSKEY RRset. For instance, publish each of the 100 DNSKEYs using two or three different signing algorithms.
- **Effect**:
    1. Each key might produce *multiple RRSIGs* for the very same RRset (e.g., RSA-SHA256, RSA-SHA512, ECDSA-P256).
    2. This yields not just 100, but *200+ RRSIGs* in a single reply (because each key can sign with multiple algorithms).
    3. The resolver might test them all if it's not sure which algorithms are "legitimate" for the zone.
- **Bonus Trick**: Combine multi-algorithm signing with multiple DS records referencing different algorithms. This ensures the resolver can't just ignore certain signatures.

---

3. Duplicate Key Tags / Collisions

- **How it Works**: Give multiple DNSKEYs the *same key tag*. Key tags are 16-bit values calculated from the public key data. While collisions are rare in normal practice, an attacker can *engineer collisions* by carefully crafting keys.
- **Effect**:

1. The resolver sees multiple DNSKEYs that match the same DS record by key tag.
2. It might then attempt *multiple signature verifications* (one for each colliding DNSKEY) to see which actually validates the zone data.
3. This results in repeated cryptographic checks, especially if each key uses a large RSA modulus or an expensive algorithm.

---

4. Cross-Signed or “Chained” RRSIGs

- **How it Works**: Instead of having each DNSKEY sign just the DNSKEY RRset, you can create a scenario where:
  - Key A signs DNSKEY set *and* separately signs an RR that references Key B.
  - Key B also signs DNSKEY set *and* references Key C, and so on.
- **Effect**:
  1. The resolver may interpret these cross-signatures as multiple potential validation paths.
  2. If the zone’s parent DS records reference *some subset* of these keys, the resolver may systematically explore each cross-sign path before giving up.
  3. This can force the resolver to do far more validations than the simplistic “one RRSIG per key” approach.

---

5. Intentional Partial Failures or Mixed Validity

- **How it Works**: Publish a set of 100 DNSKEYs, but only a *subset* (e.g., 10 of them) are valid, while the other 90 have bad signatures. Or half the signatures are expired, half are future-dated, etc.
- **Effect**:
  1. A resolver that does naive or fallback-based validation might re-check or iterate through many invalid keys and signatures before stumbling onto the valid ones.

2. It might attempt multiple cryptographic verifications per RRSIG, especially if it tries different DNSKEYs or re-checks the DS–DNSKEY match.
3. This approach specifically exploits resolvers that don't short-circuit quickly after seeing a single valid signature or a single invalid path.

---

6. Multiple RRSIGs per *Each* Individual DNSKEY

- **How it Works**: Let's say you have 100 DNSKEY records. Then for each DNSKEY, you publish *multiple RRSIGs over that single key record*.
- **Effect**:
  1. Normally, you see a single RRSIG covering the DNSKEY RRset or the same name. But if each DNSKEY is itself signed by multiple other keys (or is re-signed with multiple algorithms), you can bloat the total count of RRSIGs in the response.
  2. If each DNSKEY is covered by, say, *10 different RRSIGs*, you get 1,000 RRSIGs in total.
  3. A resolver might check each RRSIG in turn, especially if it has to confirm which key in the set is the correct signer or if DS records in the parent zone reference each potential signer.

---

7. Mix of Delegation (DS) and In-Zone Signing (DNSKEY)

- **How it Works**: Combine a large child DNSKEY set with:
  - A *sub-delegation* that also has its own large DNSKEY set, cross-signed or partially validated by the parent zone.
  - The parent zone referencing multiple DS records for each delegation.
- **Effect**:
  1. The resolver is forced to validate the child's DNSKEY set *and* an additional "sub-child" or sibling zone's DNSKEY set, each with potentially dozens of keys and signatures.
  2. If queries jump between these related subdomains, the resolver may re-validate overlapping but distinct sets of keys, drastically increasing total signature checks.

3. This can be orchestrated to cause repeated partial fetches and multiple chain-of-trust validations.

---

8. Repeated or Overlapping TTLs

- **How it Works**: Assign extremely *short TTLs* (like 1 second) to the DNSKEY RRset. Attackers can then query the resolver repeatedly, ensuring the DNSKEY set (and the associated 100+ RRSIGs) are *never properly cached*.
- **Effect**:
    1. Even if the resolver validated everything 5 seconds ago, it's forced to do it again due to TTL expiration.
    2. The overhead from re-fetching and re-validating can be compounded if multiple names or subdomains in the zone share the same bloated DNSKEY set.
    3. Combine this with any of the above methods (multiple DS, multi-alg, partial failures, etc.) to produce a *massive* number of validations over time.

Putting It All Together

Imagine a scenario that leverages *all of the above*:

1. **100 DNSKEYs** published, each with **multiple RRSIGs** for different algorithms.
2. **Multiple DS records** in the parent zone referencing every one of those keys.
3. Some keys produce **valid** signatures, some produce **expired** or *not yet valid* signatures.
4. The zone sets **very short TTLs**, ensuring minimal caching.
5. An attacker's sub-subdomain also references some of these keys in a cross-signed manner, doubling the chain-of-trust validations.

In such a "perfect storm," a resolver could be triggered to do *hundreds or even thousands* of cryptographic checks just to validate a single query's DNSKEY data. Each new query (or cache

expiry) repeats the cycle, severely burdening the resolver's CPU and potentially leading to denial of service.

Conclusion

While "100 DNSKEYs = 100 validations" is already bad enough, creative zone configurations can *dramatically* increase the total number of cryptographic checks. By manipulating DS records, multi-algorithm signing, key-tag collisions, partial failures, and more, an attacker can force resolvers to perform multiple signature validations *per key*, turning a single large DNSSEC response into a small *avalanche* of CPU-bound verification work. This is precisely how DNSSEC's inherent complexity can be exploited for resource-exhaustion or denial-of-service attacks if resolvers are not designed (or configured) to detect and mitigate such pathologies.

3. Duplicate Key Tags / Collisions

How it Works: Give multiple DNSKEYs the same key tag. Key tags are 16-bit values calculated from the public key data. While collisions are rare in normal practice, an attacker can engineer collisions by carefully crafting keys.
Effect:
The resolver sees multiple DNSKEYs that match the same DS record by key tag.
It might then attempt multiple signature verifications (one for each colliding DNSKEY) to see which actually validates the zone data.
This results in repeated cryptographic checks, especially if each key uses a large RSA modulus or an expensive algorithm.

Focus here. Elaborate. Lets think step by step

Below is a *step-by-step* illustration of how **duplicate key tags** (i.e., DNSKEY collisions) can force a resolver to perform multiple signature verifications, thus amplifying the

computational load on the resolver. While actual implementations and behaviors may vary, this breakdown shows the logic flow in a worst-case or naïve scenario.

---

1. Attacker Crafts Multiple DNSKEYs with the Same Tag

1. **Key Tag Basics**
   - A DNSKEY's **key tag** (sometimes called a "key ID") is a 16-bit checksum-like value computed from the public key data.
   - Under normal circumstances, collisions are rare because the key tag is meant to uniquely identify each key. However, it's *feasible* (though non-trivial) for an attacker to generate multiple DNSKEYs that share the same 16-bit tag.
2. **Multiple Colliding Keys**
   - The attacker creates, for example, **5 different DNSKEY** records—each using large RSA or otherwise expensive cryptography—ensuring **all 5 share the same key tag**.
   - This requires subtle manipulation of the public key data so that the resulting 16-bit tag matches.

---

2. Parent Zone Publishes a DS Record Matching That Tag

1. **DS Record Points to Key Tag**
   - In DNSSEC, the parent zone publishes **DS (Delegation Signer)** records pointing to the child's DNSKEY.
   - The DS record includes the *key tag* and *algorithm* fields (and a digest over the DNSKEY).
   - An attacker controlling the child zone also *convinces or configures* the parent zone to publish a DS record that references the same key tag (the one shared by all 5 malicious DNSKEYs).
2. **Single DS, Multiple Potential DNSKEYs**

- Because the DS record references **only that tag**, the resolver expects to find exactly one matching DNSKEY in the child zone.
- Instead, it sees 5 DNSKEYs with that *identical* tag. Each one is *potentially* the correct key that chains up to the DS.

---

3. Resolver Fetches the Child Zone's DNSKEY RRset

1. **DNSKEY RRset with Collisions**
   - When a client queries some record under the attacker's domain, the recursive resolver eventually fetches (or re-fetches) the DNSKEY RRset from the malicious child zone.
   - The DNSKEY RRset contains **all 5 colliding DNSKEYs** (plus any other keys the attacker added).
2. **Associated RRSIGs**
   - The DNSKEY RRset might be signed by *some or all* of these keys, producing multiple RRSIGs.
   - Potentially, the attacker can have each of the 5 keys generate a valid RRSIG (or produce partial/invalid signatures to further confuse the resolver).

---

4. Matching DS to DNSKEY: Multiple Candidates

1. **Resolver Logic**
   - The resolver sees the DS record in the parent zone referencing key tag "X."
   - In a *normal* scenario, there should be a *single* DNSKEY with key tag "X," so the resolver just uses that to validate signatures.
   - In this malicious scenario, there are 5 DNSKEYs with key tag "X." The resolver *doesn't know* which one is the "real" one.
2. **Possible Algorithms**

- If the DS and DNSKEYs also share the same algorithm number, the resolver is forced to consider *all 5 keys* as valid candidates.
- If some keys are intentionally mislabeled or partially invalid, the resolver might still test them until it either finds a valid chain or exhausts them all.

---

5. Repeated Signature Validation Attempts

1. **Trying Each DNSKEY**
   - The resolver picks the first DNSKEY (with tag "X") and checks whether its RRSIG over the DNSKEY RRset is valid *and* whether it matches the DS digest in the parent.
   - If that fails, it tries the second DNSKEY. If that fails, it moves on to the third… and so on.
   - In a *worst-case*, the resolver does **5 (or more) cryptographic validations** just to find a single correct chain—if there *is* a correct chain at all.
2. **Mixed Valid/Invalid Signatures**
   - The attacker can further complicate the process by making some signatures valid, some invalid, and some partially expired.
   - A naïve resolver might retry or do partial checks repeatedly, increasing CPU load.

---

6. Large or Expensive Cryptography = Big CPU Impact

1. **Big RSA Keys**
   - If these are 2048-bit or 4096-bit RSA keys, each signature validation is relatively CPU-heavy.
   - Doing 5 validations in a row might be up to *5 times* more CPU usage for this single step.
2. **Multi-Algorithm Twist**
   - If each key is also published under *multiple algorithms* (e.g., RSA-SHA256, RSA-SHA512), each key might produce *multiple signatures*. The resolver could try even

more validations per key.

---

7. Repetition with Short TTLs or Frequent Queries

1. **Short TTL Trick**
   - If the zone sets a very short TTL (e.g., 1 second) on the DNSKEY RRset, the resolver must re-fetch and re-validate all 5 keys every time the TTL expires.
   - Attackers can force the resolver to do these 5 validations repeatedly by sending multiple queries or using random subdomains to bypass the cache.
2. **Aggregate CPU Load**
   - Over many queries (or across many resolvers), the cumulative CPU overhead becomes large, effectively creating a denial-of-service condition.

---

8. Final Outcome: Amplified “Key Tag Collision” Attack

1. **Unnecessarily Many Validations**
   - The resolver ends up doing *multiple cryptographic checks* for *every single resolution attempt*, just to figure out which DNSKEY is the right one (if any).
   - In the worst-case scenario, the resolver can’t short-circuit early because there’s more than one potential match. It systematically tests each key and signature, chewing CPU cycles.
2. **Denial of Service (DoS)**
   - If the query volume is high enough (or if the attacker orchestrates additional complexities like multi-algo or partial failures), the resolver’s CPU usage skyrockets.
   - Legitimate requests may go unanswered, resulting in a *DoS* for any users relying on that resolver.

Summary of the Key Tag Collision Logic

1. **Colliding Keys**: Attacker forges multiple DNSKEYs that produce the same 16-bit tag.
2. **Parent DS**: Parent zone publishes a DS referencing the colliding tag, making all colliding keys *candidates*.
3. **Resolver Steps**:
    - Fetch DNSKEY RRset → sees multiple keys with the same tag → attempts to validate each.
    - Potentially multiple RRSIG verifications for each key (due to multi-algo or partial validity).
4. **Result**: Each query triggers *repeated cryptographic checks*, drastically elevating CPU usage.
5. **Amplification**: Short TTLs, large RSA keys, and repeated queries magnify the DoS potential.

Takeaway

By engineering **duplicate key tags** and aligning them with the *same DS record*, an attacker can force resolvers to iterate through multiple cryptographic checks to find the “correct” (or possibly nonexistent) key–signature match. This technique serves as a powerful multiplier for DNSSEC-based computational complexity attacks, leveraging the fact that resolvers *must* attempt to validate any DNSKEY that looks like a valid DS match.

5. Repeated Signature Validation Attempts

Trying Each DNSKEY

The resolver picks the first DNSKEY (with tag “X”) and checks whether its RRSIG over the DNSKEY RRset is valid and whether it matches the DS digest in the parent.

If that fails, it tries the second DNSKEY. If that fails, it moves on to the third… and so on.

In a worst-case, the resolver does 5 (or more) cryptographic validations just to find a single correct chain —if there is a correct chain at all.

Mixed Valid/Invalid Signatures

The attacker can further complicate the process by making some signatures valid, some invalid, and some partially expired.

A naïve resolver might retry or do partial checks repeatedly, increasing CPU load.

So how many max validations in the case of 100 keys and 100 signature? with this idea?

2/2

Below is a concise explanation of how, in the most pathological scenario with **100 DNSKEYs and 100 corresponding RRSIGs**, a resolver could be driven to perform **up to 10,000** individual signature validation checks (or even more if partial failures cause retries).

The Naïve Worst-Case Logic

1. **100 DNSKEYs**
   - Each DNSKEY has the same key tag or otherwise all appear to be valid candidates (e.g., due to multiple DS records referencing all of them).
2. **100 RRSIGs**
   - Each RRSIG claims to sign the DNSKEY RRset (or some relevant RRset), but the resolver doesn't know which DNSKEY actually created which signature.
3. **Trying Each DNSKEY Against Each RRSIG**
   - In a naïve (or maliciously forced) validation process, the resolver sees RRSIG #1 and tries all 100 DNSKEYs before concluding which one (if any) actually matches.
   - Then it moves on to RRSIG #2, tries all 100 DNSKEYs again, and so on through RRSIG #100.
4. **Multiplying the Checks**

$$100\ \text{RRSIGs} \times 100\ \text{DNSKEYs} = 10{,}000\ \text{signature validations}$$

5. **Mixed Valid/Invalid Signatures**
    - If many signatures are invalid, expired, or partially correct, the resolver may do repeated checks per RRSIG before giving up. In other words, **10,000** is just the baseline; partial-failure logic can push it higher.

Why 10,000 Validations Can Happen

- **Key Tag Collisions**: If the parent's DS record(s) references the same key tag, the resolver must consider *every* DNSKEY with that tag a viable signer.
- **All Keys Sign the Same RRset**: In principle, each of the 100 keys can produce a unique RRSIG over the same RRset.
- **No Early Exit**: A naive resolver might not stop after finding one valid signature; it systematically checks all possibilities.
- **Short TTLs / High Query Rate**: Attackers can force the resolver to repeat these steps frequently, leading to *continuous* massive CPU load.

Bottom Line

In a **purely naive, worst-case** scenario, 100 DNSKEYs plus 100 RRSIGs can force **up to 10,000 cryptographic verifications** on each cache miss or forced re-validation cycle. If the attacker adds further complications—multiple algorithms, partial validity, repeated timeouts—the effective number of checks can climb even higher, making this a potent vector for a DNSSEC-based denial-of-service attack.